NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007: Data Structures & Algorithms

Binary Search Trees



Presented by Dr Owen Noel Newton Fernando

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This lecture introduces binary search trees, which organise values using a clear ordering rule. We'll examine how this structure guides searching, insertion and deletion while preserving the BST property.



Objectives

- Introduce the properties and ordering rules of binary search trees.
- Explain how to identify the minimum and maximum values.
- Explain searching and insertion operations in binary search trees.
- Examine the three cases involved in deleting nodes.
- Explain how the predecessor or successor supports deletion.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This lecture develops a practical understanding of how a binary search tree stores and manages ordered data. We'll begin with the rules that distinguish it from an ordinary binary tree.

From there, we'll use those rules to locate minimum and maximum values without exploring unrelated branches. Searching and insertion will show how one comparison determines the next subtree we need to examine.

Deletion needs more care because removing a node can affect the links connecting the remaining structure. We'll separate deletion into three cases and see why a predecessor or successor allows us to handle a node with two children.

Intended Learning Outcomes

By the end of this lecture, students should be able to:

- Explain and apply the ordering rules of a binary search tree.
- Find the minimum and maximum values in a binary search tree.
- Trace and implement search and insertion operations.
- Distinguish between the three deletion cases.
- Apply the correct procedure to delete a node while preserving the binary search tree property.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



By the end of the session, you should be able to explain the ordering rule and apply it consistently at every node. That includes recognising whether a diagram represents a valid binary search tree.

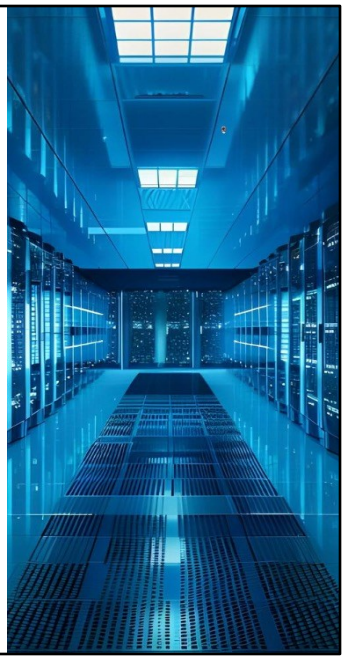
You should also be comfortable tracing the routes used to find extreme values, search for an item and insert a new one. When reading code, connect each comparison with the branch selected in the diagram.

Deletion combines several ideas, so you'll need to identify the correct case before changing any links. The final goal is to delete a requested item while ensuring every remaining subtree still satisfies the binary search tree property.

Contents

- Introduction to Binary Search Tree
- Minimum and Maximum Values
- Searching in Binary Search Trees
- Inserting Nodes into Binary Search Trees
- Deleting Nodes from Binary Search Trees
- Summary

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Here is the route we'll follow through the lecture, beginning with the structure and its ordering rule. We'll then see how that order reveals the minimum and maximum values.

The middle part concentrates on searching and insertion, using diagrams alongside recursive Python functions. Try to predict each comparison before looking at the highlighted branch or line of code.

The final section deals with deletion, where the number of children determines the procedure we use. We'll finish by bringing the operations together and reviewing how each one maintains a valid binary search tree.

Introduction to Binary Search Tree

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We'll begin by examining what makes a binary search tree different from an ordinary binary tree. Understanding its ordering rule will help us follow the searching, insertion and deletion operations introduced later.

What Is a Binary Search Tree?

- A binary search tree is an ordered data structure.
- Each node contains a value and references to at most two child nodes.
- The arrangement of the values supports efficient searching, insertion and deletion.
- Each subtree must also follow the binary search tree ordering rules.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted



A binary search tree is a binary tree with an ordering rule applied throughout the complete structure. Each node stores an item and references to no more than two children.

The arrangement of items supports directed searching because a comparison indicates which subtree could contain the target. Insertion and deletion use the same information to choose positions and preserve the organisation of the tree.

Notice that the rule must hold recursively, not only between the root and its immediate children. Every subtree must itself form a valid binary search tree because one misplaced value could direct a later operation incorrectly.

Binary Search Tree Properties

For every node in a binary search tree:

- All values in the left subtree are smaller than the current node's value.
- All values in the right subtree are greater than the current node's value.
- The left and right subtrees must also be binary search trees.
- Duplicate values are not allowed in this implementation.
- Ordering rule:
  - `Left subtree values<Current node value<Right subtree values`

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

For any node, every value in its left subtree must be smaller than the value stored at that node. Every value in its right subtree must be larger, and both subtrees must follow the same rule.

This implementation does not allow duplicate values, which keeps every comparison and insertion decision unambiguous. Other implementations may define a policy for duplicates, but we will not use one here.

Think about checking a tree from the bottom upwards rather than inspecting only each parent-child pair. A value deep in the left subtree must still be smaller than its ancestor, so local-looking connections can hide a global ordering error.

Ordered Structure of a Binary Search Tree

- In-order traversal visits the values in ascending order.
- The minimum value appears at the leftmost node.
- The maximum value appears at the rightmost node.
- Each comparison determines whether an operation continues through the left or right subtree.
- The tree must preserve its ordering property after every insertion or deletion.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

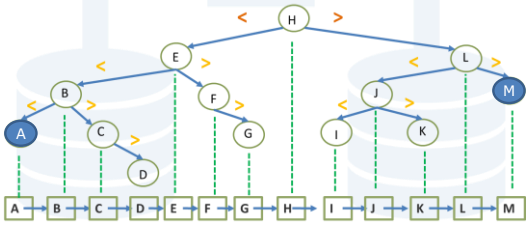
The ordered structure gives us several useful consequences without requiring a separate algorithm for each one. An in-order traversal visits every value from smallest to largest because it processes the left subtree before the node and right subtree.

The same ordering places the minimum at the leftmost reachable node and the maximum at the rightmost one. We find either extreme by following one direction until the corresponding child reference becomes None.

Comparisons also guide search and insertion through only one subtree at each step. Whatever operation we perform, its final links and values must preserve this arrangement so later operations continue to behave correctly.

Ordered Structure of a Binary Search Tree (cont.)

- The binary-search-tree property guarantees that:
  - The **minimum** is located at the **leftmost** node
  - The **maximum** is located at the **rightmost** node



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The diagram makes the extreme-value rule easier to see than a definition alone. Starting at the root, repeatedly following left links eventually reaches A, which is the minimum value in this tree.

Following right links instead reaches M, the maximum, because no larger value can appear on its left. We stop when the next reference in the chosen direction is None, not simply when we reach a leaf.

A node could still have a child on the opposite side and remain the required extreme. This leftmost and rightmost reasoning will return during deletion, when we search for a predecessor or successor within a particular subtree.

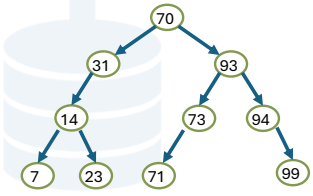
Binary Search Tree: Searching

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

With the BST ordering property established, we can now use it to search for a value. Each comparison allows us to choose either the left or right subtree and ignore the other branch.

Binary Search Tree: Searching

- Searching a BST follows a **decrease-and-conquer** approach.
- Compare the target with the current node.
- Use the BST ordering to eliminate one subtree.
- Continue searching only in the subtree that may contain the target.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Searching a binary search tree follows a decrease-and-conquer strategy because each comparison removes one complete subtree from consideration. We solve only the smaller subproblem that could still contain the target.

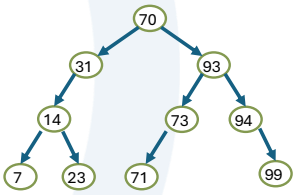
For example, if the target is smaller than the current value, everything in the right subtree is already too large. There is no reason to search that branch, so we continue only through the left child.

This differs from a general binary tree, where the absence of ordering may force us to explore both sides. The algorithm reduces unnecessary work by using information that an unordered binary tree does not provide.

Binary Search Tree: Searching

- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node):
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This recursive function expresses the search logic with three possible outcomes at each call. First, a None current node means the search has passed the point where the value could appear, so it returns False.

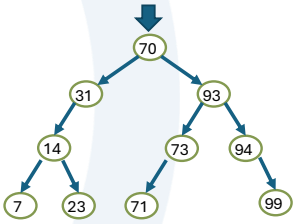
If the target equals the current item, the function returns True immediately. Otherwise, the comparison determines whether the next recursive call receives the left child or the right child.

Each call works with the same target but a different subtree root. The call stack records the route automatically, and each earlier call eventually returns the Boolean result produced by the successful match or failed search.

Binary Search Tree: Searching

- Searching for the Value **31** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 31
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Consider a search for 31, beginning at the root value 70. Since 31 is smaller than 70, the function selects the left subtree and makes a recursive call with node 31.

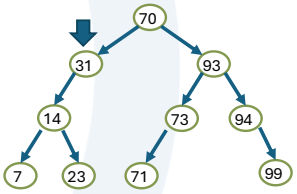
The highlighted code corresponds to this comparison, while the arrow in the tree shows the movement between nodes. We do not inspect the right subtree rooted at 93 because every value there exceeds 70.

This first decision demonstrates how the ordering rule reduces the search space. Before advancing the slide, predict what the next call returns when the current node itself contains the target value 31.

Binary Search Tree: Searching

- Searching for the Value **31** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 31
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The recursive call has now reached node 31, so the equality condition succeeds. At this point, the function returns True without examining either child of node 31.

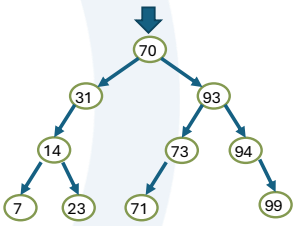
That result travels back to the earlier call at node 70, which also returns True to the original caller. The search route contains only the nodes needed to establish that the item exists.

You may have noticed that the function checks for None before accessing the node's data. This ordering prevents an error when a search reaches an empty reference and gives the unsuccessful case a clear stopping condition.

Binary Search Tree: Searching

- Searching for the Value **100** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 100
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This time the target is 100, which does not appear in the displayed tree. At node 70, the target is larger, so the search continues through the right child, node 93.

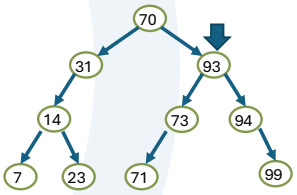
The left subtree can be ignored because all its values are smaller than 70. The recursive structure remains unchanged, but the target now produces a different route through the tree.

An unsuccessful search still follows valid comparisons rather than checking nodes randomly. We continue until the chosen child is None, which proves that 100 cannot occupy another position while the BST ordering remains valid.

Binary Search Tree: Searching

- Searching for the Value **100** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 100
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At node 93, the comparison again sends the search to the right because 100 is larger than 93. The function therefore calls itself with node 94 as the new current node.

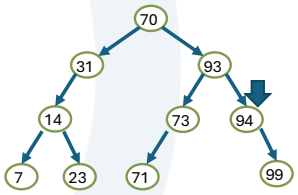
Notice how each decision rules out more than the single node we just examined. Everything to the left of 93 is also smaller than 93, so none of those values can equal 100.

This step-by-step elimination is useful when debugging a search implementation. If one relational operator points to the wrong child, the function may report False even when the target exists elsewhere in the tree.

Binary Search Tree: Searching

- Searching for the Value **100** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 100
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The current node is now 94, and 100 is still larger, so the search follows the right link once more. That link leads to node 99 in the displayed structure.

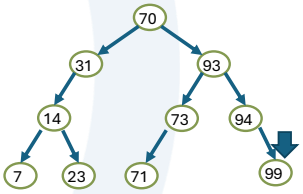
We have not found a match yet, but every previous choice remains justified by the ordering rule. Values discarded on the left of the route cannot be candidates because each is below the comparison value at that stage.

Keep an eye on the stopping condition as we approach the edge of the tree. Reaching a leaf does not end the search unless the target matches or the required next child is absent.

Binary Search Tree: Searching

- Searching for the Value **100** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 100
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At node 99, the target 100 remains larger, which means only the right subtree could contain it. The function makes another recursive call using current_node.right.

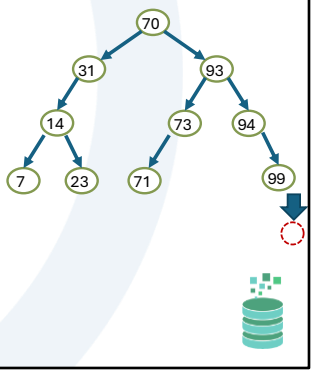
In this tree, node 99 has no right child, so the argument supplied to the next call is None. This is how an unsuccessful search moves beyond the last possible insertion position.

The diagram distinguishes visiting node 99 from testing the empty reference after it. We cannot return False merely because 99 differs from 100, since a valid larger descendant might otherwise exist on its right.

Binary Search Tree: Searching

- Searching for the Value **100** in a Binary Search Tree
- The search function should return **True** if the value is found. Otherwise, it should return **False**.

```
1 def _search(self, data, current_node): # data = 100
2         if current_node is None:
3             return False
4         elif data == current_node.data:
5             return True
6         elif data < current_node.data:
7             return self._search(data, current_node.left)
8         else:
9             return self._search(data, current_node.right)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The final recursive call receives None, so the first condition returns False. That result travels back through the calls for nodes 99, 94, 93 and 70 until it reaches the original caller.

No backtracking into alternative branches is required because the ordering comparisons already ruled them out. The empty right link after 99 identifies exactly where 100 would be inserted if we later added it.

This trace also shows why the base case must appear before any access to current_node.data. A well-designed recursive search handles a missing value safely and returns one clear Boolean result.

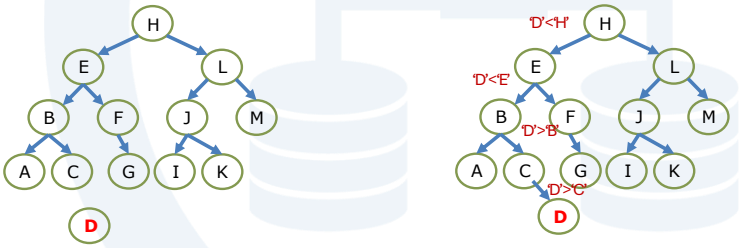
Binary Search Tree: Insertion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

The same comparison process used for searching also guides insertion. This time, when we reach an empty child reference, we create a new node there while preserving the BST ordering property.

Binary Search Tree: Insertion

- After inserting a node, the tree must remain a valid BST.
- Duplicate values are not allowed in this implementation.
- The BST ordering determines a unique insertion position.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

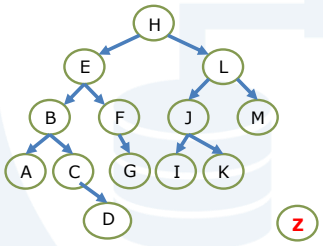
Suppose we want to insert D into the tree rooted at H. We compare D with H, then E, then B and finally C, following the appropriate child after each result.

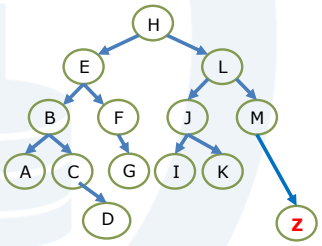
Because D is greater than C and C has no right child, that empty link becomes D's unique insertion position. Placing D anywhere else would violate at least one comparison established by its ancestors.

The implementation rejects duplicate items, so equality does not create a second node in this version. Once the new link is established, the complete structure remains a binary search tree.

Binary Search Tree: Insertion

- After inserting a node, the tree must remain a valid BST.
- Duplicate values are not allowed in this implementation.
- The BST ordering determines a unique insertion position.





© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

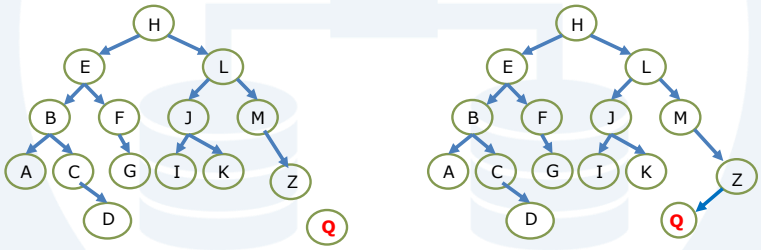
Insertion follows the same comparison process as searching, but this time we continue until we find an empty child position. The new value is placed there so that the ordering of the binary search tree remains valid.

In this example, we want to insert Z, starting with a comparison at the root node H. Since Z is greater than H, we move right to L; it is also greater than L, so we continue right to M.

Node M has no right child, which gives us the correct empty position for the new node. We therefore create node Z and assign it as the right child of M, while every existing link in the tree remains unchanged.

Binary Search Tree: Insertion

- After inserting a node, the tree must remain a valid BST.
- Duplicate values are not allowed in this implementation.
- The BST ordering determines a unique insertion position.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

After inserting Z, we insert Q by starting again from the root, H. Since Q is greater than H, we move to the right child, L, and then continue to M because Q is also greater than L.

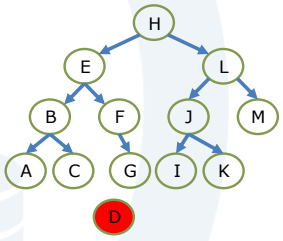
Q is greater than M, so we follow M's right child and reach node Z. This time, the direction changes because Q comes before Z in alphabetical order, which means Q must be placed in Z's left subtree.

Node Z currently has no left child, so the insertion process stops at this empty reference. We create a new node containing Q and set Z.left to Q, making Q the left child of Z while preserving the binary search tree ordering property.

Binary Search Tree: Insertion

Let's go through the insertion function step by step, assuming that we are inserting D into the given binary search tree.

```
1   def _insert(self, data, current_node):
2       if data < current_node.data:
3           if current_node.left is None:
4               current_node.left = Node(data)
5           else:
6               self._insert(data, current_node.left)
7       else:
8           if current_node.right is None:
9               current_node.right = Node(data)
10          else:
11              self._insert(data, current_node.right)
12
13
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The recursive insertion function begins by comparing the new item with the current node's item. A smaller item uses the left side, while a larger permitted item uses the right side.

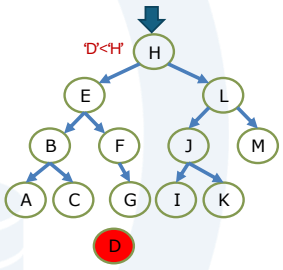
Within the selected direction, the function checks whether the child reference is None. If it is empty, a new node is created there, but if a child exists, recursion continues from that child.

For our example, we are inserting D into the displayed tree. Follow the highlighted lines and tree position together because each recursive call changes current_node while the item being inserted remains D.

Binary Search Tree: Insertion

Let's go through the insertion function step by step, assuming that we are inserting D into the given binary search tree.

```
1  def _insert(self, data, current_node):
2          if data < current_node.data:
3              if current_node.left is None:
4                  current_node.left = Node(data)
5              else:
6                  self._insert(data, current_node.left)
7          else:
8              if current_node.right is None:
9                  current_node.right = Node(data)
10             else:
11                 self._insert(data, current_node.right)
12
13
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The first comparison is between D and the root, H. Since D comes before H, the function takes the left branch and examines H's left child.

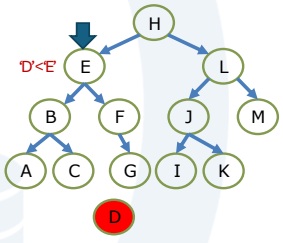
That child is E rather than None, so the algorithm cannot insert D at this level. It makes a recursive call with E as the new current node, carrying the same item D forward.

This step also establishes a lasting constraint on the final position. Wherever D is eventually attached, it must remain inside H's left subtree because every item there must be smaller than H.

Binary Search Tree: Insertion

Let's go through the insertion function step by step, assuming that we are inserting D into the given binary search tree.

```
1  def _insert(self, data, current_node):
2          if data < current_node.data:
3              if current_node.left is None:
4                  current_node.left = Node(data)
5              else:
6                  self._insert(data, current_node.left)
7          else:
8              if current_node.right is None:
9                  current_node.right = Node(data)
10             else:
11                 self._insert(data, current_node.right)
12
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At node E, D is again smaller than the current item, so the function selects the left child. E already has B on that side, which means recursion continues rather than creating a node.

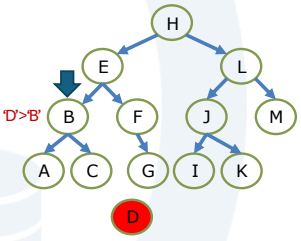
The route now reflects two constraints: D must be smaller than H and also smaller than E. Both conditions remain true for every location within the subtree we are entering.

Notice that the function does not restart from the root after each comparison. Each call narrows the problem to one subtree until an appropriate child reference becomes empty.

Binary Search Tree: Insertion

Let's go through the insertion function step by step, assuming that we are inserting D into the given binary search tree.

```
1  def _insert(self, data, current_node):
2          if data < current_node.data:
3              if current_node.left is None:
4                  current_node.left = Node(data)
5              else:
6                  self._insert(data, current_node.left)
7          else:
8              if current_node.right is None:
9                  current_node.right = Node(data)
10             else:
11                 self._insert(data, current_node.right)
12
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The current node is B, and this comparison changes direction because D is greater than B. The function therefore examines B's right child instead of continuing left.

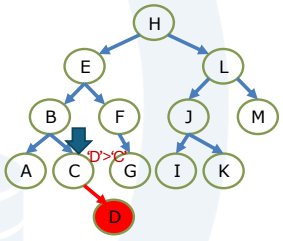
That right child contains C, so the algorithm makes another recursive call with C as the current node. D's eventual position must be greater than B while still satisfying the upper limits imposed by E and H.

This is why checking only the immediate parent is insufficient when validating a BST. The complete route carries constraints from every ancestor, although recursion enforces them naturally through its comparisons.

Binary Search Tree: Insertion

Let's go through the insertion function step by step, assuming that we are inserting D into the given binary search tree.

```
1  def _insert(self, data, current_node):
2          if data < current_node.data:
3              if current_node.left is None:
4                  current_node.left = Node(data)
5              else:
6                  self._insert(data, current_node.left)
7          else:
8              if current_node.right is None:
9                  current_node.right = Node(data)
10             else:
11                 self._insert(data, current_node.right)
12
```



C.right is None → create Node('D')

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At node C, D is greater, so the function checks the right child of C. This time the reference is None, which means we have reached the unique valid insertion position.

The function creates a new node containing D and assigns it to current_node.right. No further recursive call is needed because the new node has been attached successfully.

Trace the path once more as H, E, B, C and then D. Every choice follows the ordering rule, and all existing links remain unchanged, so the resulting structure is still a valid binary search tree.

Binary Search Tree: Deletion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Deletion requires more care because removing a node may affect the connections to other nodes. We'll examine three cases based on whether the node has zero, one or two children.

Binary Search Tree: Deletion

Deleting a node from a binary search tree requires us to preserve the BST ordering property. The procedure depends on the number of children the node has:

- **Case 0: No children**
  Remove the leaf node directly.
- **Case 1: One child**
  Replace the node with its only child.
- **Case 2: Two children**
  Replace the node's value with its in-order predecessor or successor, and then remove the predecessor or successor from its original position

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The deletion procedure begins by locating the requested item while keeping track of the relevant parent link. What happens next depends entirely on whether the node has zero, one or two children.

For a leaf, the parent's link becomes None, while for one child, that link changes to the child. With two children, disconnecting the node directly would lose two subtrees, so we first copy a predecessor or successor item.

After copying the replacement, we remove its original node using the simpler leaf or one-child case. Identifying the case before changing any reference is the safest way to reason about the implementation.

Deletion Case 0 — Node with No Children

When the node to delete is a leaf:

- The node has no children.
- Remove the node from the binary search tree.
- Set the corresponding child reference in its parent to None.
- No other nodes need to be moved.
- The binary search tree property remains unchanged.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

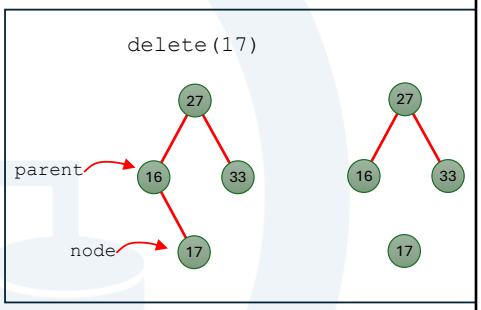
Case 0 is the simplest situation because the node to delete has no children. Removing its incoming link cannot disconnect another part of the tree.

We identify whether the leaf is the parent's left or right child, then set that specific reference to None. If the leaf is also the root and the only node, the root reference itself becomes None.

No values need to move, and no subtree needs to be reattached. In Python, the disconnected object becomes eligible for memory reclamation after the function releases any remaining references to it.

Deletion Case 0: Node with No Children

- Locate node 17 in the BST.
- Save item 17 for returning later.
- Confirm that node 17 has no children.
- Identify its parent, node 16.
- Since 17 is the right child, set 16.right = None.
- Node 17 becomes unreachable and is automatically deallocated after all references are released.
- The remaining BST structure stays unchanged.
- Return the deleted item, 17.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

In this example, the requested item is 17, located as the right child of node 16. We save 17 first because the deletion operation is designed to return the item that was removed.

Node 17 has no children, so this follows Case 0. Setting 16.right to None disconnects the leaf without changing node 16 or any other connection in the tree.

A local variable may still refer to the old node until the function finishes, but it no longer belongs to the BST. After those references are released, Python manages its memory, and the function returns the saved item 17.

Deletion Case 1 — Node with One Child

When the node to delete has one child:

- Identify the node's only child.
- Connect the parent of the node being deleted directly to that child.
- Remove the node from the binary search tree.
- The child and its subtree move into the deleted node's position.
- The binary search tree property remains unchanged.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

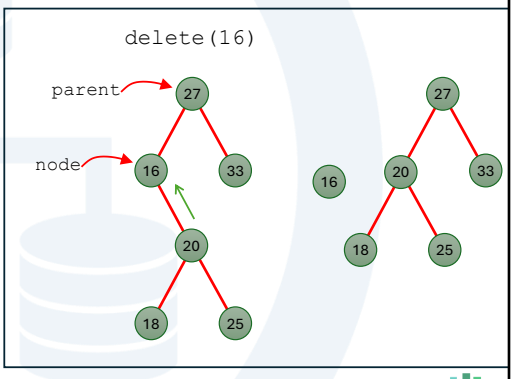
Case 1 applies when the node to delete has exactly one child. We preserve the child and everything below it by connecting the parent directly to that child.

This link update bypasses the deleted node while keeping the complete descendant subtree reachable. If the node being deleted is the root, the root reference changes directly to its only child instead.

The ordering property remains valid because every item in the child's subtree already occupied the correct side of the deleted node and its parent. No item within the promoted subtree needs to be copied or visited.

Deletion Case 1 — Node with One Child

- Find node 16, save its item, and identify its parent, node 27.
- Identify its only child, node 20.
- Set 27.left = 20 to bypass node 16.
- Node 20 and its subtree remain connected.
- Node 16 is disconnected from the BST.
- Python reclaims node 16 after all remaining references to it are released.
- Return the saved item, 16.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here we delete node 16, whose only child is node 20. We first save item 16 and identify node 27 as the parent whose left link currently points to the node being removed.

Assigning 27.left = 20 bypasses node 16 and keeps node 20's complete subtree connected. Nodes 18 and 25 require no individual updates because their links remain inside the subtree rooted at 20.

Node 16 is now outside the BST, although a temporary local variable may still reference it. Once those references are released, Python handles reclamation, and the operation returns the saved item 16.

Deletion Case 2 — Node with Two Children

When the node to delete has two children:

- Find either the in-order predecessor or the in-order successor.
- The predecessor is the largest value in the left subtree.
- The successor is the smallest value in the right subtree.
- Copy the predecessor's or successor's value into the node being deleted.
- Delete the predecessor or successor from its original position using **Case 0 or Case 1**.
- Either approach is valid. In the following examples and implementation, we use the in-order successor.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

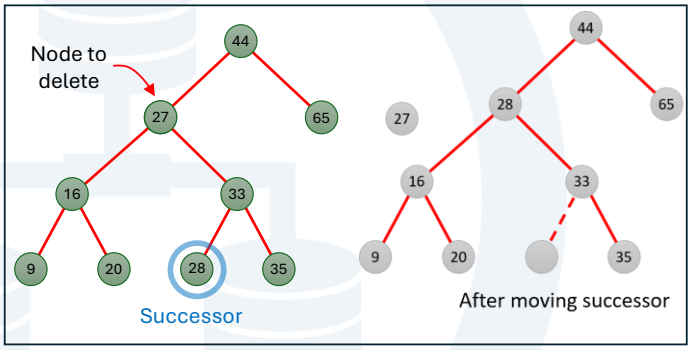
Case 2 applies when the node to be deleted has both a left and a right child, so we cannot simply remove or bypass it without affecting the two subtrees. Instead, we choose a suitable replacement item that preserves the BST ordering property.

The in-order predecessor is the largest item in the left subtree, while the in-order successor is the smallest item in the right subtree. We copy the chosen item into the node being deleted, while keeping its existing child links unchanged.

We then delete the original predecessor or successor from its previous position using Case 0 or Case 1, because it can have at most one child. Either approach is valid, but in the following examples and implementation, we will use the in-order successor.

Deletion Case 2: Successor Is a Leaf Node

- Find node 27 and save its item for returning later.
- Since node 27 has two children, find its successor, node 28.
- Copy item 28 into the node currently storing 27.
- The same node now stores 28; its links to 16 and 33 remain unchanged.
- Recursively delete the original successor node 28.
- Since the successor is a leaf, set 33.left = None.
- Return the originally deleted item, 27.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

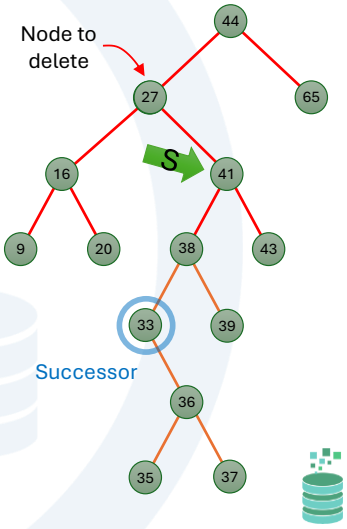
The item to delete is 27, and the node has two children, so we use Case 2. Before changing anything, we save 27 because that is the item the outer deletion call must return.

The smallest item in the right subtree is 28, making it the in-order successor. We copy 28 into the existing node, whose links to 16 and 33 remain exactly as they were.

The original successor node is a leaf under node 33, so recursive deletion uses Case 0 and sets 33.left to None. This removes the temporary duplicate, while the outer operation returns the saved item 27.

Deletion Case 2: Successor Is a Non-Leaf Node

- **Save the item to delete, 27, for returning later.**
- **Start at the right child: S = node.right.**
- **Move S left while it has a left child.**
- Follow the path 41 → 38 → 33.
- Node 33 is the in-order successor.
- Copy item 33 into the node containing 27.
- Recursively delete the original node 33 using **Case 1**.
- Since node 33 has one child, set 38.left = 36.
- The subtree rooted at 36 remains connected.
- Return the originally saved item, 27.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

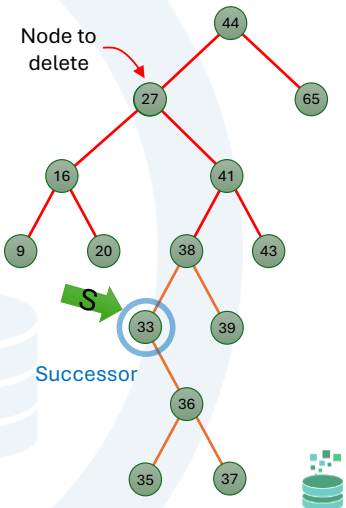
This example again deletes a node with two children, but we will work through the process one step at a time. The node to delete contains 27, so we first save the item 27 because the deletion function must return it later.

Since node 27 has both a left and a right child, we cannot simply disconnect it or bypass it. We therefore need to find its in-order successor, which must be located somewhere within the right subtree of node 27.

To begin that search, we create a pointer called S and set S = node.right. As shown by the highlighted arrow, S now points to node 41, and the next slide will show how we move this pointer to locate the successor.

Algorithm to Find the Successor

- Save the item to delete, 27, for returning later.
- Start at the right child: S = node.right.
- Move S left while it has a left child.
- **Follow the path 41 → 38 → 33.**
- **Node 33 is the in-order successor.**
- Copy item 33 into the node containing 27.
- Recursively delete the original node 33 using **Case 1**.
- Since node 33 has one child, set 38.left = 36.
- The subtree rooted at 36 remains connected.
- Return the originally saved item, 27.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

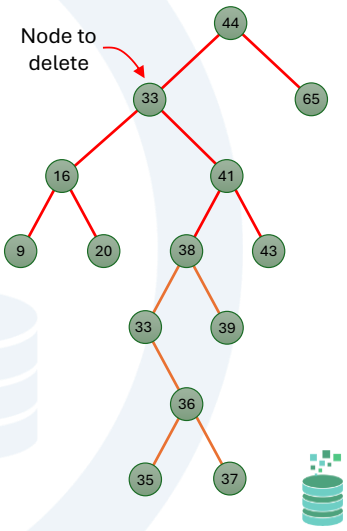
Pointer S currently starts at node 41, the right child of the node containing 27. To find the in-order successor, we repeatedly move S to its left child while another left child exists.

From node 41, S first moves left to node 38, which is smaller than 41 but still greater than 27. Node 38 also has a left child, so S moves left once more and reaches node 33.

Node 33 has no left child, so the search stops at this position. Therefore, node 33 is the smallest node in the right subtree of 27 and becomes its in-order successor, ready for the replacement step shown on the next slide.

Algorithm to Find the Successor

- Save the item to delete, 27, for returning later.
- Start at the right child: S = node.right.
- Move S left while it has a left child.
- Follow the path 41 → 38 → 33.
- Node 33 is the in-order successor.
- **Copy item 33 into the node containing 27**.
- Recursively delete the original node 33 using **Case 1**.
- Since node 33 has one child, set 38.left = 36.
- The subtree rooted at 36 remains connected.
- Return the originally saved item, 27.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

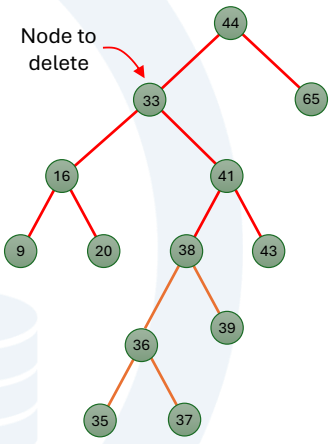
We have identified node 33 as the in-order successor, so the next step is to use its item as the replacement. We copy item 33 into the existing node that currently contains 27.

Notice that we are changing only the stored item, not moving the successor node itself. The original node keeps its existing links to node 16 on the left and node 41 on the right, so both subtrees remain connected.

At this stage, the value 27 has disappeared from the tree, but the value 33 temporarily appears in two places. The next step is therefore to delete the original successor node containing 33 from its old position, which we will examine on the following slide.

Algorithm to Find the Successor

- Save the item to delete, 27, for returning later.
- Start at the right child: S = node.right.
- Move S left while it has a left child.
- Follow the path 41 → 38 → 33.
- Node 33 is the in-order successor.
- Copy item 33 into the node containing 27.
- **Recursively delete the original node 33 using Case 1.**
- **Since node 33 has one child, set 38.left = 36.**
- **The subtree rooted at 36 remains connected.**
- **Return the originally saved item, 27.**



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

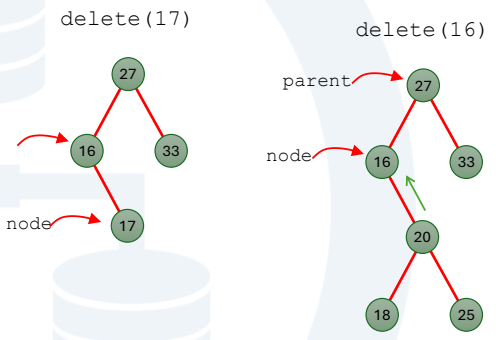
After copying item 33 into the original node, we still need to remove the old successor node containing 33. That node has one child, node 36, so the recursive deletion follows Case 1.

The parent of the original successor is node 38, whose left reference currently points to the old node 33. We therefore set 38.left = 36, bypassing the original successor and connecting node 38 directly to node 36.

The complete subtree rooted at 36, including nodes 35 and 37, remains connected without further changes. Once the original node 33 is disconnected, Python can reclaim it after all references are released, and the outer deletion call returns the item saved at the beginning, which is 27.

BST Deletion Algorithm: Cases 0 and 1

1. **Find the node**
   - Search for the item using the BST property.
   - Keep references to the node and its parent.
   - If the item is not found, return None.
   - Save the node's item to return after deletion.
2. **Case 0: Node has no children**
   - Set the parent's corresponding child link to None.
   - If the node is the root, set root = None.
3. **Case 1: Node has one child**
   - Identify the node's only child.
   - Set the parent's corresponding child link to that child.
   - If the node is the root, set root to its child.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

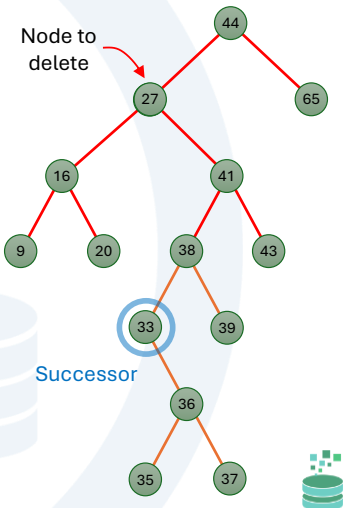
The deletion algorithm begins by searching for the item using the BST ordering property. As we move through the tree, we keep references to both the node to be deleted and its parent, because the parent's link may need to be updated. If the item cannot be found, we return None; otherwise, we save the node's item so that it can be returned after the deletion is completed.

For Case 0, the node has no children, so it can be removed without affecting any subtree. In the example, node 17 is the right child of node 16. We simply set the corresponding child link in node 16 to None, which disconnects node 17 from the BST. If the node being deleted is the root and has no children, we instead set the root to None.

For Case 1, the node has exactly one child, so we must preserve that child and its entire subtree. In the example, node 16 has only one child, node 20. We update the corresponding link in its parent, node 27, so that it points directly to node 20. This bypasses node 16 while keeping the subtree rooted at node 20 connected. If the node being deleted is the root, we simply update the root to point to its only child.

BST Deletion Algorithm: Case 2

4. **Case 2: Node has two children**
   - Find the in-order successor: the smallest node in the right subtree.
   - Copy the successor's item into the node being deleted.
   - Recursively delete the original successor node.
   - The recursive deletion uses:
     - **Case 0** if the successor is a leaf, or
     - **Case 1** if the successor has a right child.
5. **Complete the deletion**
   - The disconnected node is reclaimed after all references are released.
   - Return the originally saved item.
   - The BST ordering remains preserved.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

For Case 2, the node to be deleted has two children, so we cannot simply remove or bypass it. Instead, we find its in-order successor, which is the smallest node in its right subtree. In this example, we are deleting node 27. Starting from its right child, node 41, we repeatedly follow the left links until we reach node 33, making 33 the in-order successor.

We then copy the successor's item, 33, into the node currently containing 27. Notice that we copy only the item; we do not move the successor node itself. The original successor node must then be deleted from its current position. This recursive deletion is simpler because the in-order successor cannot have a left child, so it will fall under either Case 0, if it is a leaf, or Case 1, if it has a right child.

In this example, the original successor node 33 has one right child, node 36, so Case 1 is applied. Node 33 is bypassed and its parent, node 38, is connected directly to node 36. Once the original successor is disconnected, the deletion is complete. The originally saved item, 27, is returned, and the BST ordering property remains preserved.

Binary Search Tree: Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've covered the main ideas and operations involved in binary search trees. This final section brings those concepts together and reviews how the BST ordering property guides searching, insertion and deletion.

Conclusion

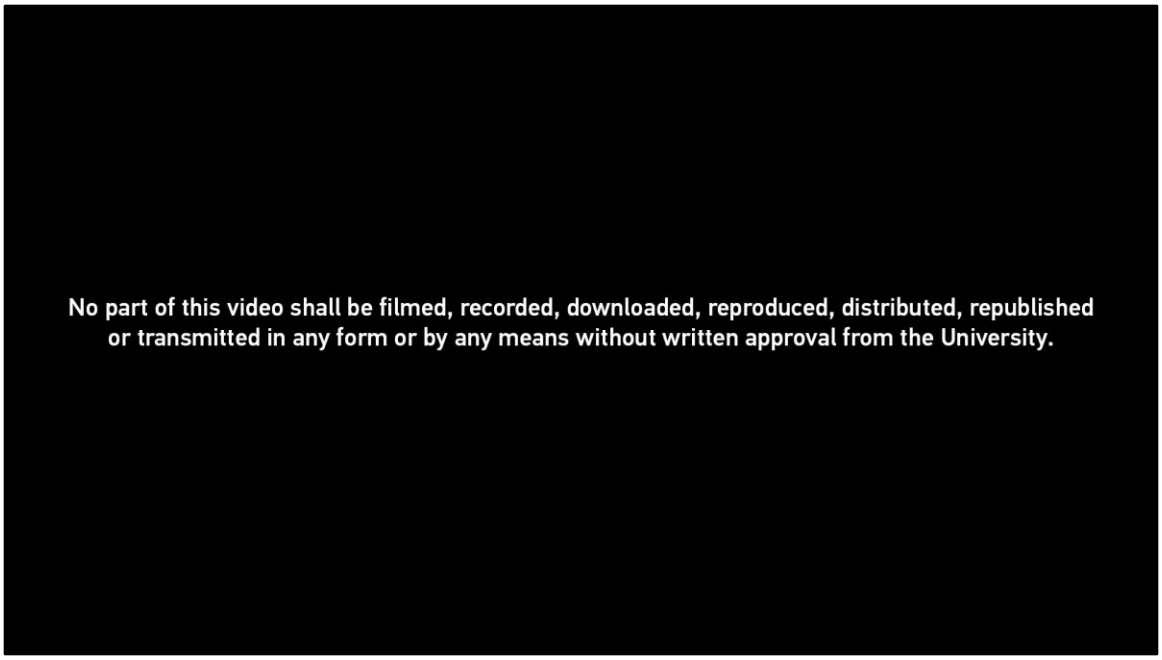
- A binary search tree stores smaller items in the left subtree and larger items in the right subtree.
- Search and insertion follow the BST ordering property.
- In-order traversal visits the values of a BST in ascending order.
- Deletion depends on whether the node has zero, one or two children.
- A node with two children uses its in-order successor as the replacement item.
- Each operation preserves the BST ordering property.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A binary search tree organises its values using one consistent ordering rule: smaller values belong in the left subtree, while larger values belong in the right subtree. This ordering property guides the main operations we have examined throughout the lecture.

When searching or inserting, we compare the target value with the current node and decide whether to continue through the left or right subtree. The same ordering also means that an in-order traversal visits the values of a binary search tree in ascending order.

Deletion requires more care because the procedure depends on whether the node has zero, one or two children. For a node with two children, we use the in-order successor as the replacement item and then delete the original successor. After each operation, the BST ordering property is preserved, ensuring that the tree remains a valid binary search tree.


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.